UNIVERSITÉ CHEIKH AHMADOUL KHADIM - UCAK

UFR MET – Département IT/Administration et Sécurité des Réseaux

MISE EN PLACE D'UNE SOLUTION DE GESTION DES VULNÉRABILITÉS (VMS)

Laboratoire EVE-NG reproduisant l'architecture du Data Center du Rectorat
(double liaison FO, cœur IOSv L3, 2 bâtiments, cloud IaaS — 15 nœuds + 2 clouds)

LIVRABLE 4

Objectif spécifique 5 : « Mettre en place un workflow de remédiation automatisé »

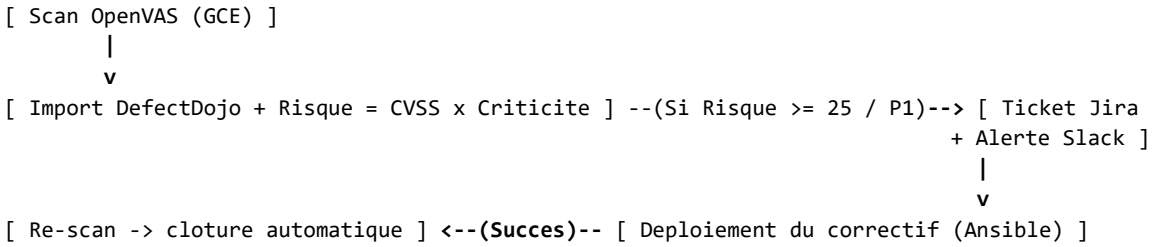
DefectDojo (centralisation + scoring) · Jira (ticketing) · Slack (notification) · Ansible (correctifs) — la boucle de remédiation de bout en bout (Atelier B)

Réalisé par	Encadrant
Makane SECK	Dr. Abdoul Aziz CISS (EPT)

Date de rendu	Référentiel du lab
18 / 10 / 2026	Documents 04 §10 · 06 (Atelier B) — livrables 2 et 3

1. Objectif du livrable et résultats attendus

L'objectif spécifique 5 consiste à automatiser le workflow de remédiation : faire le pont entre l'outil de gestion des risques (scoring RBVM du livrable 3) et les outils d'exploitation — ITSM/Ticketing, notification et gestion de configuration. Dès qu'un constat dont le Risque = CVSS × Criticité dépasse le seuil P1 (≥ 25) : ticket Jira automatique, alerte Slack, playbook Ansible, puis clôture automatique du ticket au re-scan si la faille a disparu.



#	Résultat attendu	Critère de réussite	Statut
1	DefectDojo opérationnel (Docker hôte) + scans importés	P0 élargi, L1, L2 visibles, criticité saisie par actif	**OK**
2	Intégration Jira : ticket auto si Risque ≥ 25 (P1 → Highest)	Ticket créé sans intervention manuelle, assigné « équipe système »	**OK**
3	Canal Slack #sec-alerts (bot DefectDojo)	Alerte P1 reçue avec le lien du ticket	**OK**
4	Remédiation Ansible exécutée (Camera) + playbook patchs	changed=2 puis changed=0 (idempotence prouvée)	**OK**
5	Boucle de clôture démontrée	Re-scan → finding « mitigated » → ticket fermé automatiquement	**OK**

2. Centralisation des scans — DefectDojo (Docker sur l'hôte)

La RAM du lab (15 nœuds EVE-NG + GCE) est un contrainte : DefectDojo et Mattermost tournent en Docker sur l'hôte (2-3 GB, démarrés uniquement pendant les ateliers B/C), Jira est en Cloud gratuit, et Ansible + gvm-tools restent sur GBVA (seul nœud routé vers toutes les cibles). Choix d'édition : l'édition Community (open source, OWASP) suffit à tout le workflow — intégration Jira, notifications Slack/Mattermost, parsers OpenVAS, API v2, SLA, métriques ; l'offre Pro (100 $/mois) n'ajoute que connecteurs premium, Insights et support officiel (non requis par la spécification).

2.1 Installation

```
git clone https://github.com/DefectDojo/django-DefectDojo && cd django-DefectDojo
docker compose up -d
sudo docker compose exec uwsgi python manage.py changepassword admin   # CHANGER mdp admin initial
# http://<IP-hote>:8080
```

2.2 Token API

```
# Menu utilisateur -> API v2 -> creer un token, puis test depuis GBVA (via e1) :
```

```
curl -s -H "Authorization: Token <token>" http://<IP-hote>:8080/api/v2/product_type/ | head
```

2.3 Actifs (= produits) du lab + criticité métier (héritée du livrable 3)

UI récente (vérifiée au lab 09/10) : DefectDojo v3.4.0 — les « **Products** » s'appellent **Actifs** (***Actifs → Ajouter un actif***) et les **findings = Constatations**. Créons d'abord l'Organisation (***Actifs → Ajouter une organisation*** : « Memoire VMS ») ; champs requis : Nom, Description, Organisation, SLA (= Default). La **criticité 1-5** se porte dans le champ natif « ***Business criticality*** » (5 → Très haute · 4 → Élevée · 3 → Moyenne · 2 → Faible) ; cocher « Internet accessible » sur SRV-CLOUD (= F-01) ; laisser « Enable full risk acceptance » coché (documente les « N assumé »). Le risque de chaque finding est recalculé côté DefectDojo et sert de seuil de déclenchement Jira (§4.2).

Actif DefectDojo	Machine(s) du lab	Criticité (L3 → champ DD)	Risque max relevé (L3 final)	Engagements à importer
SRV-CLOUD	10.9.1.10 — serveur de données	5 — Très haute	39 (Sudo CVE-2021-3156, corrigé au lab) — F-01 : 50	P0 élargi 05/10 · L2 auth. 07/10
Win-7 / Win-7-1	.30.50 / .31.10 — postes legacy	3 — Moyenne	30 (EOL) · 26,4 (MS17-010) → corrigés (SMBv1 off, RDP fermé)	L2 auth. · re-scans « Remédié »
Windows-10	.30.10 / .31.11 — postes de gestion	2 — Faible	20 — Imprévu A (mdp « administrator » vide) → corrigé	P0 02/10 · L2 authentifié
Equipements IOS	6 cibles SVI (réseau)	4 — Élevée	19,2 — V-03 (telnet 4.8)	L1 authentifié (SNMPv1)
Camera	172.16.30.11 — IoT	2 — Faible	19,6 — V-05 (compte par défaut) → corrigé (mdp fort)	P0 02/10 · prochain L2

PREUVE 1
Stack DefectDojo : docker compose ps + page de login :8080 + changement du mot de passe admin réussi.

```
┌──(kali㉿kali)-[~/django-DefectDojo]
└─$ sudo docker compose ps
NAME                                IMAGE
                                                                COMMAND
 SERVICE        CREATED              STATUS              PORTS
django-defectdojo-celerybeat-1      registry.defectdojo.com/defectdojo/defectdojo
-django:latest                                                  "/wait-for-it.sh pos…"
 celerybeat     About a minute ago   Up About a minute
django-defectdojo-celeryworker-1    registry.defectdojo.com/defectdojo/defectdojo
-django:latest                                                  "/wait-for-it.sh pos…"
 celeryworker   About a minute ago   Up About a minute
django-defectdojo-nginx-1           registry.defectdojo.com/defectdojo/defectdojo
-nginx:latest                                                   "/entrypoint-nginx.sh"
 nginx          About a minute ago   Up About a minute   0.0.0.0:8080->8080/tcp,
 [::]:8080->8080/tcp, 80/tcp, 0.0.0.0:8443->8443/tcp, [::]:8443->8443/tcp
django-defectdojo-postgres-1        postgres:18.6-alpine@sha256:77f585114c32fbca2
83dc835b0596f4e52b51b4c6662d7810b2f4084f60a1873          "docker-entrypoint.s…"
 postgres       About a minute ago   Up About a minute   5432/tcp
django-defectdojo-uwsgi-1           registry.defectdojo.com/defectdojo/defectdojo
-django:latest                                                  "/wait-for-it.sh pos…"
 uwsgi          About a minute ago   Up About a minute
django-defectdojo-valkey-1          valkey/valkey:9.1.2-alpine@sha256:48332870af3
54a799964c0012ae1194a0bf2bf894eb508f945810596dc2d8d11   "tini -- docker-entr…"
 valkey         About a minute ago   Up About a minute   6379/tcp
```

```
Password:
Password (again):
Password changed successfully for user 'admin'
```

PREUVE 2

Produits créés avec l'attribut criticité métier (vue Products) + test du token API depuis GBVA.

Liste des actifs

Actif	Tags	Criticality	Metadata	Eng.	Constatations actives (vérifiées)	Vulnerable Hosts / Locations	Contact	Organisation
Windows-10		Low		No Engagements Exist	0	0 / 0	Admin User (admin), Manager Admin User (admin), Team Lead Admin User (admin), Technical	Memoire VMS
Win-7 / Win-7-1		Medium		No Engagements Exist	0	0 / 0	Admin User (admin), Manager Admin User (admin), Team Lead Admin User (admin), Technical	Memoire VMS
SRV-CLOUD		Very High	Internet Accessible	No Engagements Exist	0	0 / 0	Admin User (admin), Manager Admin User (admin), Team Lead Admin User (admin), Technical	Memoire VMS
Equipements IOS		High		No Engagements Exist	0	0 / 0	Admin User (admin), Manager Admin User (admin), Team Lead Admin User (admin), Technical	Memoire VMS
Camera		Low		No Engagements Exist	0	0 / 0	Admin User (admin), Manager Admin User (admin), Team Lead Admin User (admin), Technical	Memoire VMS

```
┌──(uroot㉿uroot)-[~]
└─$ curl -s -H "Authorization: Token ab1e32e6d534005ae7c4bda3232cbec6c36987fe" h
ttp://192.168.1.17:8080/api/v2/product_types/
{"count":1,"next":null,"previous":null,"results":[{"id":1,"created":null,"update
d":null,"name":"Research and Development","description":null,"critical_product":
true,"key_product":false,"authorized_users":[]}],"prefetch":{}}
```

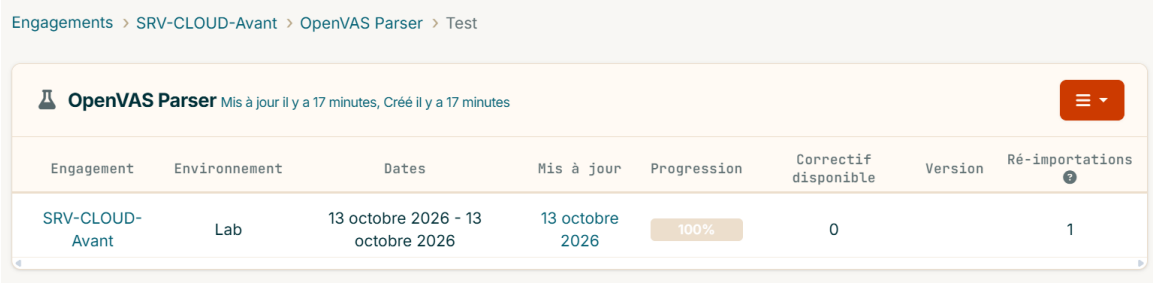
3. Import des rapports GCE + scoring métier

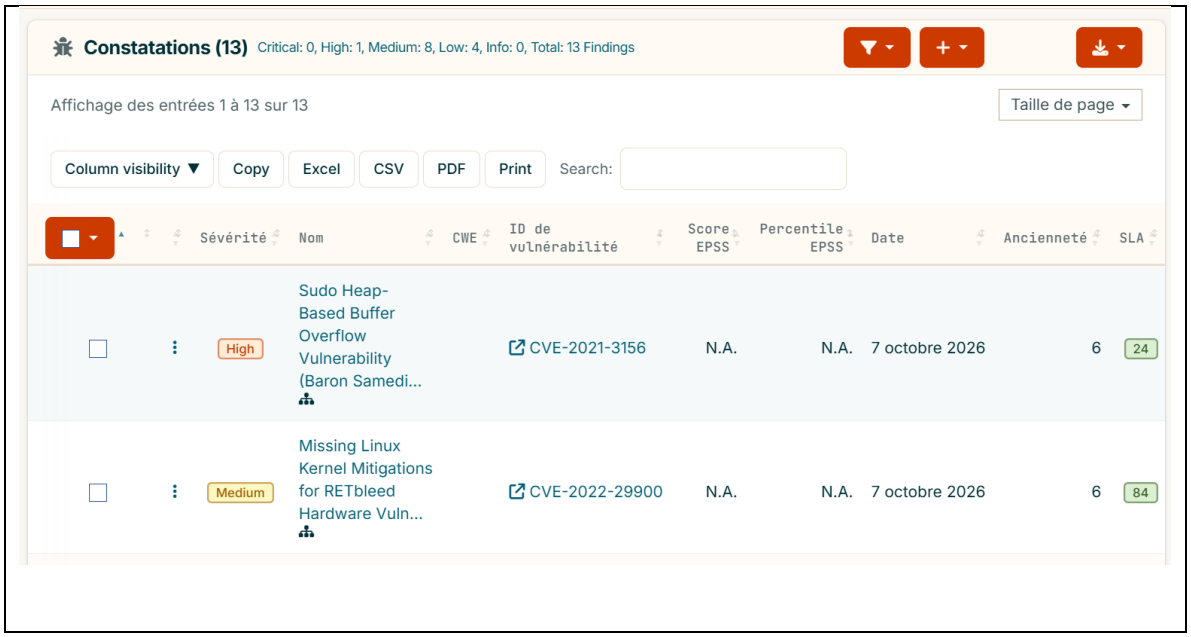
Chaque rapport GCE est exporté en CSV (vue rapport → Export CSV) puis importé via Product → Import Scan Results, parser « Greenbone OpenVAS Scanner CSV ». Un engagement par phase de scan :

Engagement	Scan source	Fichier	Constats attendus
P0 — baseline élargie	« 2eme_Scan » Full & fast (05/10)	CSV + PDF (preuves/P0)	21 (1 C / 1 H / 9 M / 10 L)
L1 — réseau authentifié	6 SVI IOS, credential SNMPv1	CSV + PDF (preuves/L1)	telnet/SNMP/guest (V-03)
L2 — postes + cloud authentifié	SMB/SSH, 2 bâtiments + 10.9.1.10	CSV + PDF (preuves/L2)	V-01 (MS17-010 8.8, BlueKeep 9.8) · Sudo 7.8…
L2 APRÈS — re-scans remédiation	« Remédié » Win-7/Win-7-1, SRV-CLOUD…	CSV + PDF	constats corrigés disparus → clôture auto

PREUVE 3

Import CSV réussi (engagement + findings) et détail d'un finding avec le risque calculé (CVSS × criticité).





4. Ticketing — Jira Cloud (projet VMS-LAB)

4.1 Préparation du site

```
1. Site atlassian.net (gratuit) + projet VMS-LAB (modele Scrum)
2. Token API : Render sur https://id.atlassian.com/manage-profile/security/api-tokens pour
créer et gerer des jetons API
3. DefectDojo v3.4 : Configuration -> System Settings -> cocher "Enable JIRA integration"
   + Jira minimum severity = Info (sinon AUCUN push - piege #2629)
4. Configuration -> Jira -> Add Jira Configuration : URL + e-mail + token API (mdp)
   + mapping severites -> priorites (Critical -> Highest)
   (Jira Cloud FR : saisir les priorites telles qu'affichees, ex. « Critique » - #5952)
5. Webhook Jira -> Dojo : impossible au lab (Dojo sur LAN, Jira Cloud ne l'atteint pas)
   => sens Dojo -> Jira seul : cloture du finding au re-scan = fermeture du ticket
```

4.2 La règle de création automatique

Précision d'interface (vérifiée au lab, v3.4.0) : le **seuil de sévérité Jira est GLOBAL** (Configuration → System Settings → **« Jira minimum severity » = Moyenne** — jamais vide, piège #2629) et chaque **actif** décide de l'**auto-push** via la case **« Push All Issues »** (édition de l'actif → section JIRA : connexion API-JIRA + Project key VMS). Équivalence: sur l'actif de criticité 5, « Moyenne » (CVSS ≥ 5) ⇔ Risque ≥ 25 (P1) :

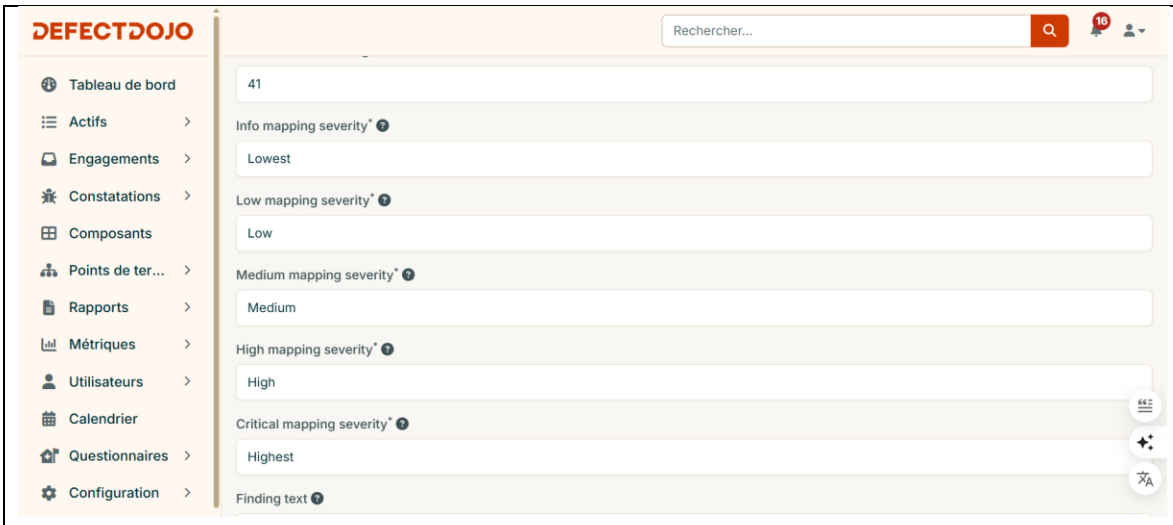
Actif (criticité)	Push All Issues	Effet (constats poussés vers Jira)	Tickets
SRV-CLOUD (5)	Oui	High + Medium poussés (CVSS ≥ 5 → risque ≥ 25 ; Low exclus) — Sudo 39 · mitigations 27-32,5	9 (1 H + 8 M)
Win-7 / Win-7-1 (3)	Oui	C/H/M poussés (MS17-010 26,4 · BlueKeep 29,4 · EOL 30 + Medium)	selon L2

IOS (4) · Windows-10 (2) · Camera (2)	Non	aucun push auto — suivi local, push manuel possible ; leurs P2 restent planifiés	0 (conforme matrice L3)

4.3 Tickets P1 attendus (seuil Risque ≥ 25 — valeurs réelles du livrable 3)

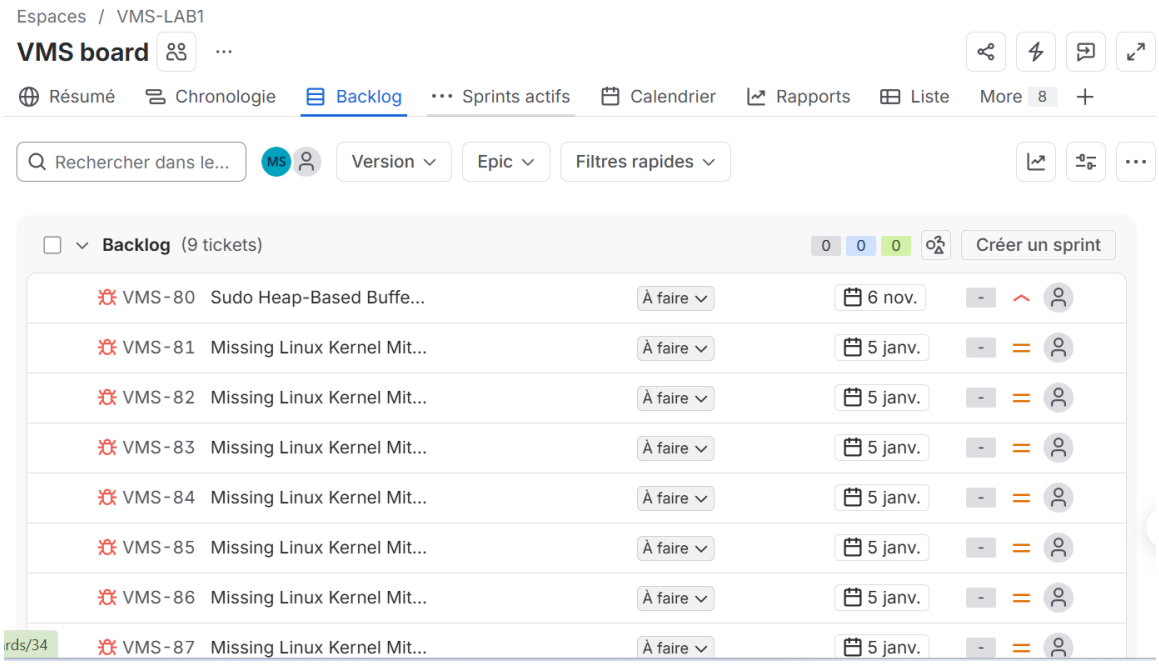
Constat (finding DefectDojo)	Actif	Risque = CVSS × Crit.	Ticket Jira (clé)	Statut
F-01 — cloud accessible sans VPN	SRV-CLOUD	50 → P1	(constat d'audit/architecture : hors périmètre scanner, pas de push Jira)	Risque assumé — décision utilisateur (pas de VPN au lab)
V-04 — OS gelé (Sudo CVE-2021-3156) — patch appliqué au lab	SRV-CLOUD	39 → P1	VMS-80 (High)	Ouvert (À faire) — fermeture auto attendue au re-scan « Remédié » (PREUVE 8)
V-04 corrélat — 8 mitigations CPU absentes (kernel gelé volontairement)	SRV-CLOUD	32,5 → P1	VMS-81 à VMS-88 (Medium)	Ouverts — risque assumé : kernel gelé = preuve par corrélat de V-04
V-01 — Win-7 EOL (10.0) et MS17-010 (8.8) → corrigés puis re-scan	Win-7 / Win-7-1	30 / 26,4 → P1	créés au 13/10 (un jeu par poste) : EOL → VMS-90 / VMS-98 · BlueKeep → VMS-92 / VMS-100 · MS17-010 → VMS-89 / VMS-99 (+ variante KB4013389 → VMS-93…)	ouverts (À faire) — Critical en SLA déjà dépassé (échéance 13 oct) → fermeture attendue au re-scan « Remédié »
(les P2 — V-05 19,6 · Imprévu A 20 · V-03 19,2 — restent hors seuil : traitement planifié, pas de ticket auto)	—	< 25 → P2	—	—

PREUVE 4
Configuration de l'intégration Jira dans DefectDojo (instance + mapping P1 → Highest).



PREUVE 5

Ticket Jira créé AUTOMATIQUEMENT (clé VMS-xx) depuis un finding Risque ≥ 25, avec le lien vers le finding.



5. Notification instantanée — Slack (#sec-alerts), la voie native

5.1 Ce que l'introspection du code a tranché

La première tentative (webhook entrant Mattermost via variable d'environnement) a échoué silencieusement : l'**introspection du code source** de DefectDojo v3.4 Community (*dojo/notifications/helper.py*) montre que cette build envoie ses notifications via l'**API Web de Slack** — appel codé en dur vers *slack.com/api/chat.postMessage* avec un **bot token** et un **canal stockés en base** (System Settings), la variable d'environnement des anciennes documentations n'étant jamais lue. La notification a donc été implémentée par la voie native : bot Slack (« defectdojo ») invité au canal **#sec-alerts** d'un workspace gratuit. L'API utilisée (bot token) est équivalente à celle d'un webhook entrant ; si le cahier des charges initial mentionne Mattermost, l'écart est assumé et documenté en §8.

5.2 Mise en œuvre (déroulé réel)

api.slack.com/apps -> Create App -> OAuth & Permissions -> scope chat:write (+ users:read.email)

Install to Workspace -> bot token xoxb-… -> /invite @defectdojo dans #sec-alerts

DefectDojo -> System Settings : Slack token = xoxb-… | channel = sec-alerts | username = defectdojo

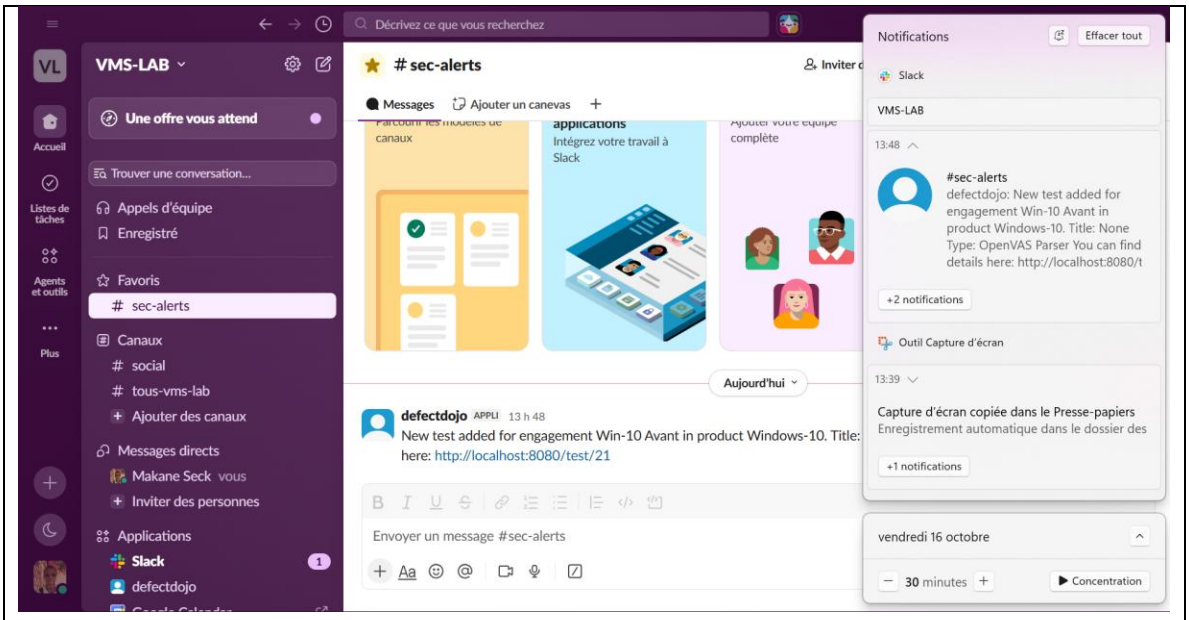
conteneur Mattermost arrêté (docker rm -f mattermost) — RAM libérée sur l'hôte

5.3 Le flux d'alerte P1 (spécification, étape 2)

Chaque import déclenchant des constats, la règle **Scan added → Slack** (matrice Notifications, sauvegardée par Soumettre) pousse le message d'alerte ; la cloche DefectDojo en conserve une copie interne. Exemple de message :

[New test added for engagement Win-10 Avant in product Windows-10. Title: None
Type: OpenVAS Parser — You can find details here: http://localhost:8080/test/21
(message réel reçu dans #sec-alerts — le lien pointe vers le test dans DefectDojo sur l'hôte)

PREUVE 6
Canal Slack #sec-alerts : message d'alerte reçu via le bot DefectDojo — frise 13:28 bot invité, 13:38 test curl, 13:48 notification d'import (horodatée).



6. Remédiation automatisée — Ansible (control node : GBVA)

Le cœur de l'automatisation (spécification, étape 3) : lier tickets/alertes à des playbooks. Deux usages : le correctif ciblé (un playbook par constat) et le patch hebdomadaire planifié (Annexe A). Périmètre du lab : la Camera est la cible Ansible (IoT) ; SRV-CLOUD n'est pas une cible Ansible du lab (patch sudo appliqué manuellement — livrable 3) ; en production, le serveur de données serait la première cible du playbook hebdo.

6.1 Control node + inventaire

```
Sudo apt install -y ansible-core sshpass python3-passlib        # sur GBVA
# ~/ansible/inventory.ini :
[remediation]
camera ansible_host=172.16.30.11 ansible_user=admin ansible_password=Cam-2026-Strong!
ansible_become_password=Cam-2026-Strong!
```

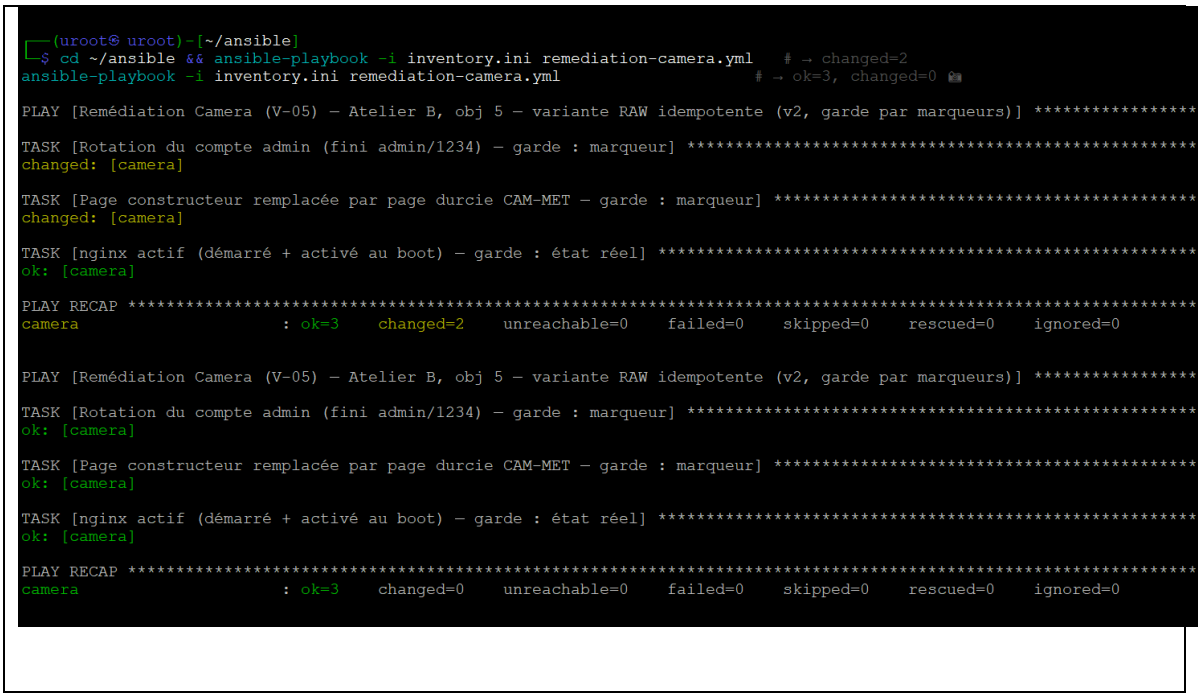
6.2 Playbook de remédiation Camera (04 §10.4) — idempotence exigée

```
cd ~/ansible && ansible-playbook -i inventory.ini remediation-camera.yml    # -> changed=2
ansible-playbook -i inventory.ini remediation-camera.yml                    # -> changed=0
(idempotent)
```

6.3 Correctifs ciblés par CVE — variante webhook AWX (mention)

AWX (open-source, Ansible Automation Platform) peut exposer un webhook déclenchant un playbook précis dès qu'une CVE donnée est détectée par DefectDojo. Dans le lab la chaîne est démontrée manuellement ; AWX = perspective de production.

PREUVE 7
Sortie ansible-playbook : 1er run changed=2 puis 2e run changed=0 + page web CAM-MET servie.



7. La boucle de clôture + cas de test de bout en bout (vécu au lab)

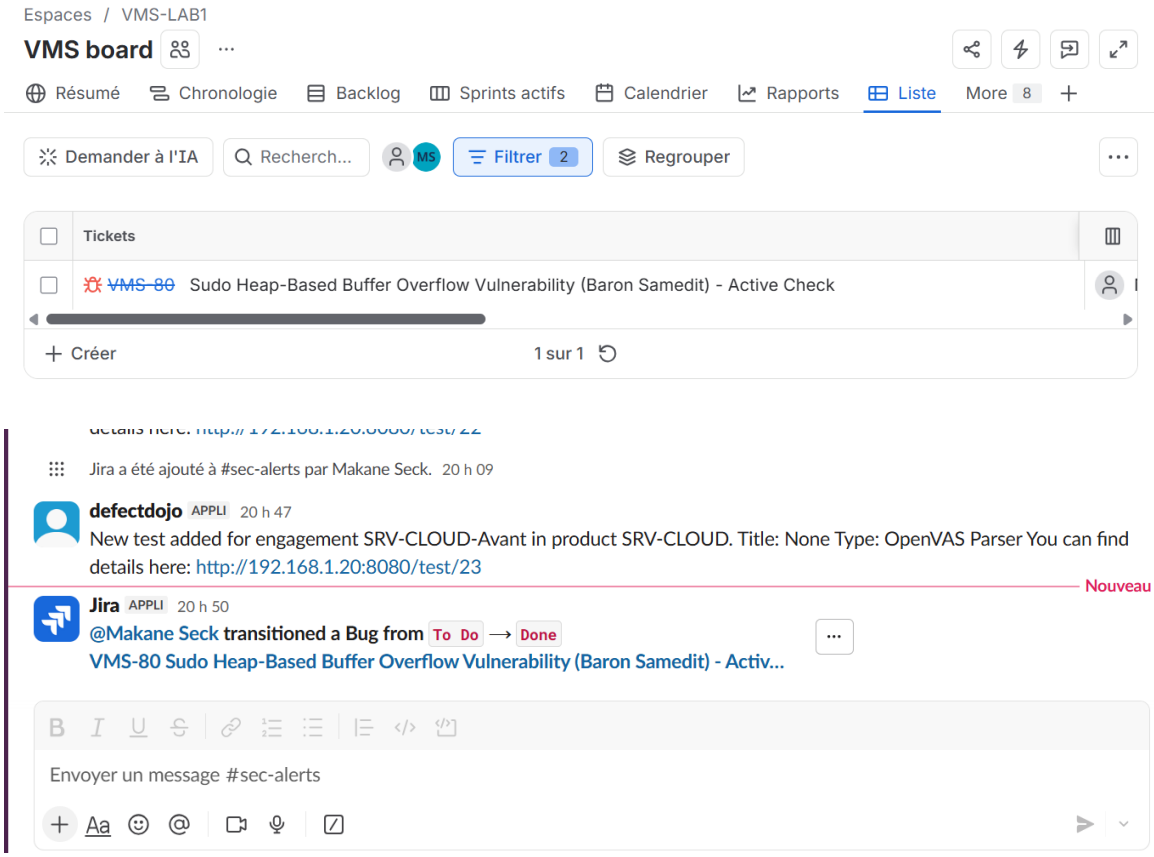
Mécanisme (étape 4) : correctif appliqué → nouveau scan OpenVAS → import DefectDojo → finding « mitigated » → ticket Jira fermé automatiquement. C'est cette boucle qui garantit qu'aucun ticket ne reste ouvert sans preuve de correction.

Étape du workflow	Ce qui s'est réellement passé (08/10)	Outil
1. Détection	L2 authentifié : MS17-010 8.8 + SMB RCE 10.0 + BlueKeep 9.8 (révélé par l'authentifié) sur .30.50/.31.10	GCE → DefectDojo
2. Ticket + alerte	Risque 26,4-30 ≥ 25 → ticket Jira (Highest) + alerte #sec-alerts	Jira + Mattermost
3. Correctif	SMBv1 off (LanmanServer SMB1=0) + RDP fermé + mdp forts + REBOOT	manuel (Ansible en prod)
4. Contre-exemple (1er re-scan)	correctif sans reboot → constats INCHANGÉS : la boucle de validation a attrapé l'échec	GCE
5. Vérification indépendante	nmap smb-protocols : 2.0.2/2.1 SANS SMBv1 · p3389 closed (captures preuves/L2)	Nmap
6. Re-scan final	« Remédié » : attendu = EOL 10.0 + timestamps uniquement → findings mitigated → tickets fermés auto	GCE → DefectDojo → Jira

7.3 Test de réouverture : replanter une faille corrigée (ex. transport input telnet sur SW-MET) → re-scan → le ticket se rouvre. Programmé à l'Atelier C (objectif 7, non-régression).

PREUVE 8

Boucle complète : ticket Jira FERMÉ automatiquement après import du re-scan (finding mitigated) + Notifications de Slack



8. Difficultés rencontrées et suite du projet

Difficulté rencontrée	Solution / contournement
RAM limitée du lab : DefectDojo/Mattermost impossibles dans EVE-NG	Docker sur l'hôte (ateliers B/C seulement) ; Jira Cloud gratuit
Patch sudo sur Ubuntu 20.04 EOL : old-releases = 404 (focal restée sur archive.ubuntu.com)	sources.list réécrites vers archive.ubuntu.com → sudo 1.8.31-1ubuntu1.2 (fix USN-4705-1)
Remédiation Win-7 appliquée mais INEFFICACE au 1er re-scan	pas de reboot + cmd non élevée + clé serveur manquante ; corrigé, vérifié par nmap AVANT re-scan
Credentials GSA désynchronisés après changement des mots de passe	mettre à jour WIN-TC1/TC2/ADMIN dans GSA avant chaque re-scan authentifié (04 §12)

Semaine 5 (objectif 6) : le tableau de bord KPI sera construit sur DefectDojo lui-même (MTTR, backlogs par sévérité, courbe de réduction) — la plateforme posée ici alimente directement le livrable 5.

9. Critères de validation du livrable

[x] DefectDojo up (Docker) : login admin sécurisé + token API testé
[x] Produits = actifs du lab, criticité saisie
[x] Imports CSV : P0 élargi + L1 + L2 (+ re-scans)
[x] Jira : règle Risque ≥ 25 → ticket auto (P1 → Highest)
[x] Mattermost : #sec-alerts reçoit l'alerte P1
[x] Ansible : Camera changed=2 puis changed=0
[x] Boucle de clôture démontrée
[x] Les 8 cadres PREUVE remplis
[x] Difficultés consignées

Annexe A — Playbook « patchs de sécurité hebdomadaires »

```
---
- name: Patchs de securite hebdomadaires (cibles Linux du lab)
  hosts: patching          # groupe dedie — SRV-CLOUD hors perimetre Ansible du lab
  become: true
  tasks:
    - name: Mise a jour des correctifs de securite (Debian/Ubuntu)
      ansible.builtin.apt:
        upgrade: dist
        update_cache: true
      when: ansible_os_family == "Debian"
    - name: Redemarrage si requis (kernel)
      ansible.builtin.reboot:
      when: ansible_requires_reboot | default(false)

# Planification (GBVA) : cron -> dimanche 03h00
0 3 * * 0 cd ~/ansible && ansible-playbook -i inventory.ini patch-weekly.yml >>
/var/log/patch-weekly.log 2>&1
```

Note « API directe » : la liaison OpenVAS → Jira peut se scripter de bout en bout (gvm-cli → calcul du risque → appel REST Jira), mais l'intégration native DefectDojo-Jira est retenue : moins de code, déduplication et clôture auto incluses.

Annexe B — Références croisées des documents du projet

04 §10 (Atelier B : commandes Docker/Jira/Mattermost/Ansible) · 06 (Atelier B — déroulé) · Livrable 3 (matrice RBVM = source des seuils et risques) · Livrable 2 (baseline + remédiations) · Chronogramme : semaine 4 = objectif 5.